

初級編 単一GPUプログラミング (OpenACC, C/C++)

次世代HPC・AI研究開発支援センター(HAIRDESC)

資料本編前に

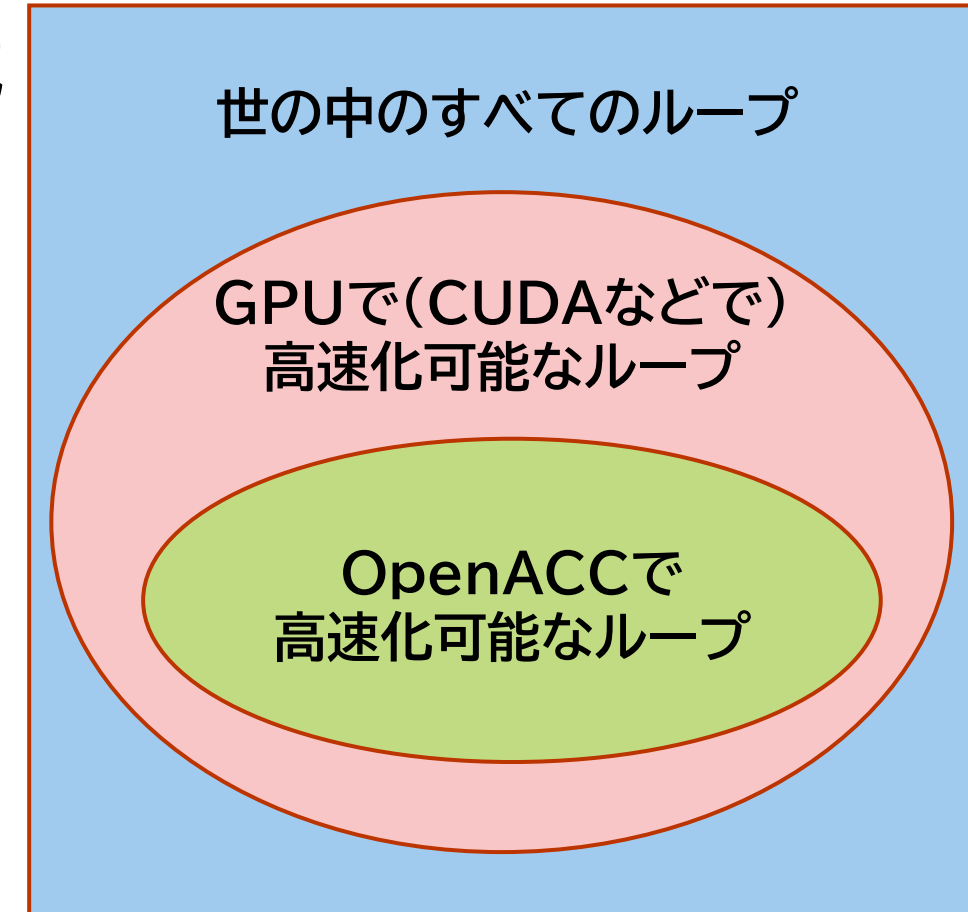
- 本資料は, OpenACCを用いてGPU化を開始する方向け
- OpenMPを用いたマルチコアCPU向けのスレッド並列については理解していることを前提とする
- プログラミング言語としてはC/C++を用いる
- OpenACCの規格としては, 2.7に準拠する
 - 標準的なコンパイラ(NVIDIA HPC SDK)であれば, 20.7以降のすべてのリリースでサポートされている

Section 1.

実行イメージ

GPUにおけるループ並列化

- GPU における高速化は通常, プログラム中の重たいループ構造を並列化することで達成
- 今回学ぶOpenACCは**特定のループ構造を簡単に並列化**できる
 - 全てのループ構造を並列化できるわけではない
 - どのようなループなら並列化可能か知る必要がある
- 主に以下の3つを記述できる
 - どの処理をGPUで実行するか
 - (GPUで実行する領域内に出てくる)ループが, データ独立か, リダクションか, それ以外か
 - どこでデータを移動するか



OpenACCで並列化できるループ

データ独立な
ループの例

```
for ( i = 0; i < N; i++)
  C[i] = A[i] + B[i];
```

リダクションの必要な
ループの例

```
sum = 0;
for ( i = 0; i < N; i++)
  sum += A[i];
```

データ依存のある
ループの例

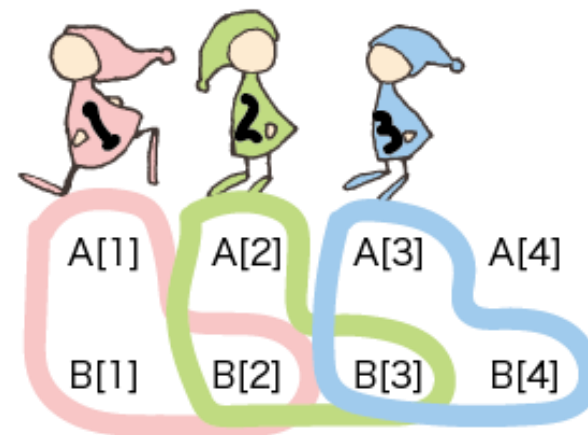
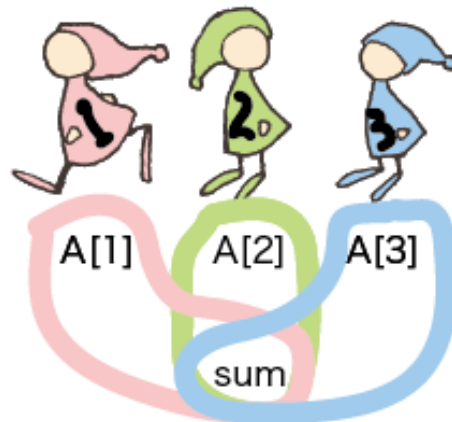
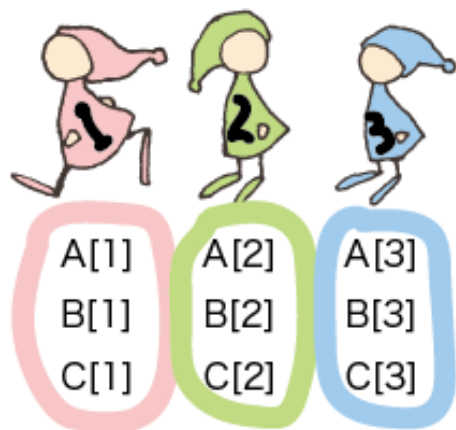
```
B[0] = 0;
for ( i = 0; i < N; i++)
  B[i+1] = B[i] + A[i];
```

OpenACCで簡単に並列化できる

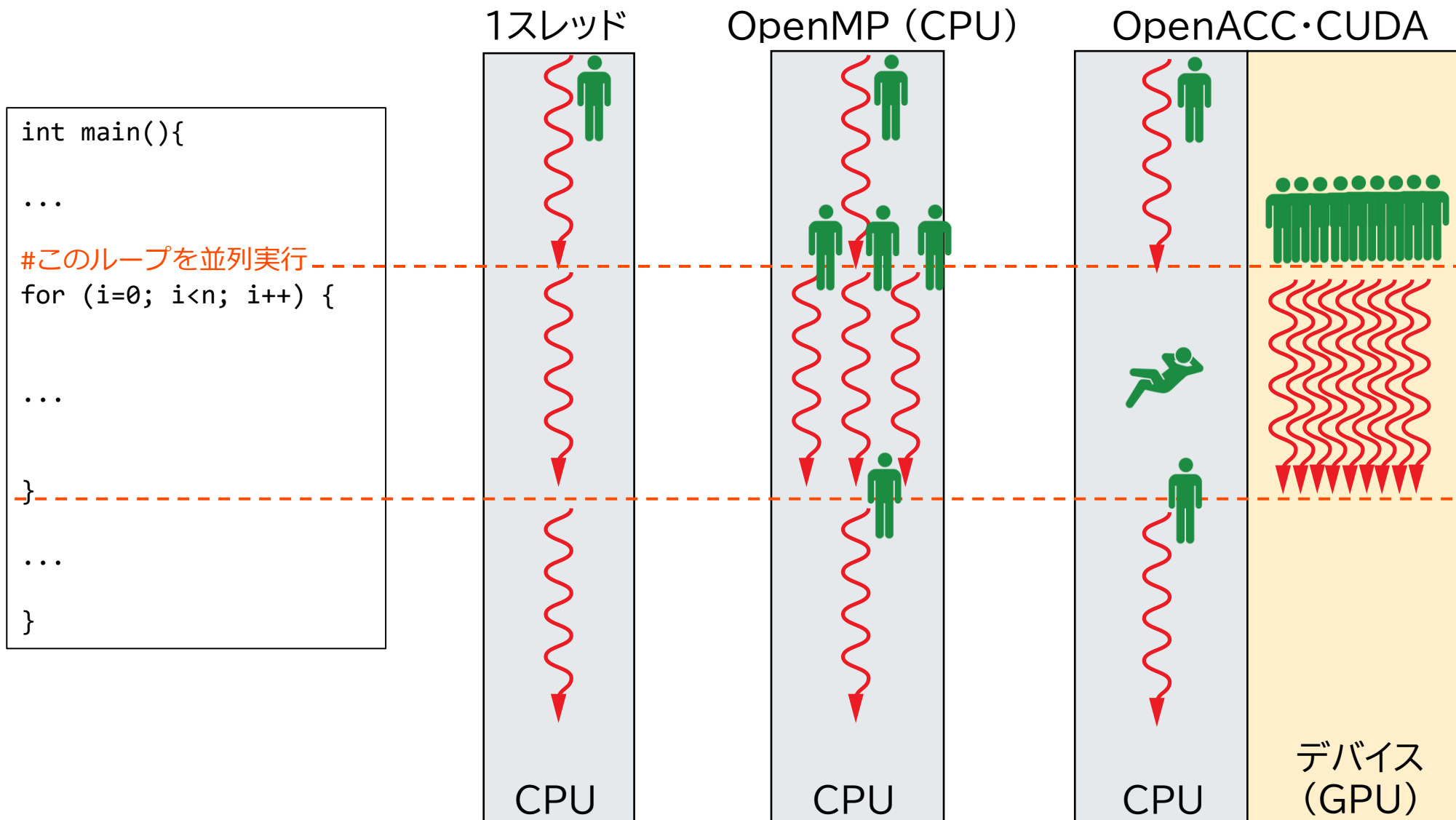
※OpenACCでも、GPUで正しく動くコードを書くことはできる
しかし、遅いので意味がない

CUDAでも比較的簡単に並列化できる

CUDAで高速実装可能だが難しい (shared memory や warp shuffle を駆使する必要がある)



OpenACCの実行イメージ



Section 2.

GPU化方針・実装方法

指示文でのGPU化方針

1. まずは演算部分のGPU実装のみに注力

- (マルチコアCPU向けの)OpenMP実装されていれば,
`#pragma omp parallel for` をGPU向けの指示文に置き換えていく
- メモリ上のデータがCPU側にあるか, GPU側にあるかは一旦忘れて実装
- Unified/Managed Memoryという機能(後述)に,
GPU上のメモリ確保, CPU-GPU 間のデータ転送は全てお任せ

2. (Unified/Managed Memory 実装での性能に満足できない場合) data/update指示文を使ってコードをアップデート

- Managed Memory では, 一旦メモリを読んで, ページフォルトがあればハードウェアレベルでページ単位で転送という仕組み
- 必要なデータ転送は自分で指示した方が必然的に速くなる
- GPUDirect系の機能を活用する際には,
GPU上にデータが置かれていることを保証できる実装手法を使いたくなる

指示文でのGPU化方針

1. まずは演算部分のGPU実装のみに注力

- (マルチコアCPU向けの)OpenMP実装されていれば,
`#pragma omp parallel for` をGPU向けの指示文に置き換えていく
- メモリ上のデータがCPU側にあるか, GPU側にあるかは一旦忘れて実装
- Unified/Managed Memoryという機能(後述)に,
GPU上のメモリ確保, CPU-GPU 間のデータ転送は全てお任せ

2. (Unified/Managed Memory 実装での性能に満足できない場合) data/update指示文を使ってコードをアップデート

- Managed Memory では, 一旦メモリを読んで, ページフォルトがあればハードウェアレベルでページ単位で転送という仕組み
- 必要なデータ転送は自分で指示した方が必然的に速くなる
- GPUDirect系の機能を活用する際には,
GPU上にデータが置かれていることを保証できる実装手法を使いたくなる

ループ並列化に使用する構文

- parallel構文

- GPU側で並列処理する領域を(ユーザーが)明示的に指示するための構文
- 並列可能なループであることをユーザーが保証(自己責任による並列化)
 - #pragma omp parallel for がついていたようなループであれば問題なくGPU化できる
→ OpenMP並列されたCPUコードの移植では, まずは parallel 構文を試してみる
 - 計算結果が不正だった際には, kernels構文に切り替えてデバッグに活用するなど
- 並列化対象のループを指示するために, loop指示文も併用

- kernels構文

- GPU側で並列処理できそう・速くなりそうな領域をコンパイラに伝えるための構文
- コンパイラが並列化可能性などを調査・判定(コンパイラの実験による並列化)
- GPU化されない場合もあり, loop指示文や指示節の追加による補助が必要な時も

- loop構文

- 実際にGPU側で並列化する処理(for文など)を指定

ループ並列化(parallel構文による実装)

- `#pragma omp parallel for` を置き換えていく
 - スレッド並列化可能なループであること目印
- `#pragma acc parallel` はGPU化可能だとユーザーが保証(自己責任)
 - 対象リージョンはGPU化される
 - 元々`#pragma omp parallel for`がついていたループであれば、スレッド並列によるGPU化が可能と考えて問題ない
 - `kernels`ではGPU化されなかったが、本来GPU化できるはずという時に使う
- `#pragma acc parallel` 単体ではGPU化領域の指定のみなので、`#pragma acc loop` もつけて並列化対象のループを指示する

```
#pragma omp parallel for  
for(int i = 0; i < N; i++){  
    // main computation  
}
```



```
#pragma acc parallel  
#pragma acc loop  
for (int i = 0; i < N; i++) {  
    // main computation  
}
```

ループ並列化(kernels構文による実装)

- `#pragma omp parallel for` を置き換えていく
 - スレッド並列化可能なループであること目印
- `#pragma acc kernels` はGPUで並列化するかコンパイラが最終判断
 - GPU化はするが逐次実行になることもある
(並列化できない場合, 性能が出ないと予想される場合)
- `#pragma acc kernels` だけでもGPUで並列化してくれるが,
`#pragma acc loop` もつけることが多い(最適化などの追加指示に使う)

```
#pragma omp parallel for
for(int i = 0; i < N; i++){
    // main computation
}
```



```
#pragma acc kernels
for(int i = 0; i < N; i++){
    // main computation
}

#pragma acc kernels
#pragma acc loop
for(int i = 0; i < N; i++){
    // main computation
}
```

最適化などの追加指示例

- parallel/kernels 構文での違いは特にない
- 各処理が独立に実行できることを保証
 - loop指示文にindependentを追加
 - コンパイラが並列性を十分に見きれなかった場合
- 多重ループの結合
 - loop指示文にcollapse(n)を追加
 - 大外ループだけでは並列度が足りない場合
 - Tightly-nested loopでないといけない
- リダクション演算
 - loop指示文にreduction(...)を追加
 - CPU向けOpenMPと同じ

```
#pragma acc parallel
#pragma acc loop independent
  for (int i = 0; i < N; i++) {
    // main computation
  }

#pragma acc parallel
#pragma acc loop collapse(3)
  for (int i = 0; i < Nx; i++) {
    for (int j = 0; j < Ny; j++) {
      for (int k = 0; k < Nz; k++) {
        // main computation
      }
    }
  }

#pragma acc parallel
#pragma acc loop reduction(+ : sum)
  for (int i = 0; i < N; i++) {
    // main computation
    sum += val[i];
  }
```

指示文でのGPU化方針

1. まずは演算部分のGPU実装のみに注力

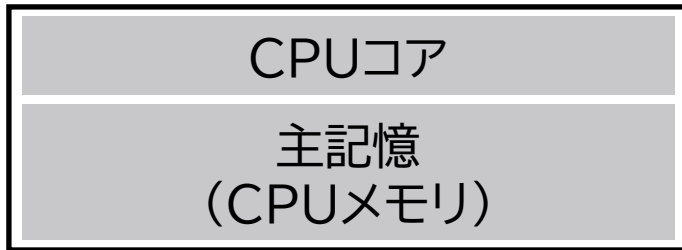
- (マルチコアCPU向けの)OpenMP実装されていれば,
#pragma omp parallel for をGPU向けの指示文に置き換えていく
- メモリ上のデータがCPU側にあるか, GPU側にあるかは一旦忘れて実装
- Unified/Managed Memoryという機能(後述)に,
GPU上のメモリ確保, CPU-GPU 間のデータ転送は全てお任せ

2. (Unified/Managed Memory 実装での性能に満足できない場合) data/update指示文を使ってコードをアップデート

- Managed Memory では, 一旦メモリを読んで, ページフォルトがあればハードウェアレベルでページ単位で転送という仕組み
- 必要なデータ転送は自分で指示した方が必然的に速くなる
- GPUDirect系の機能を活用する際には,
GPU上にデータが置かれていることを保証できる実装手法を使いたくなる

CPUとGPUのメモリ空間

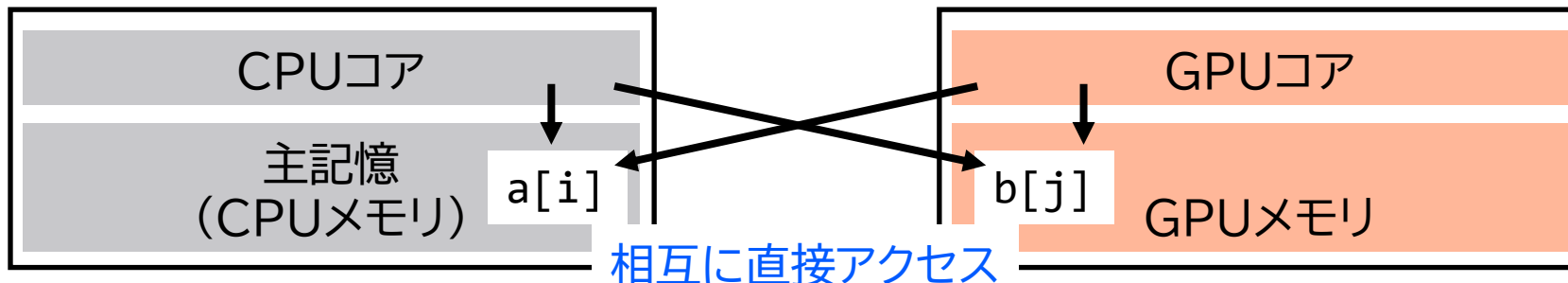
- CPUとGPUは、物理的に独立したメモリを持っていることが多い
 - GPU上で処理するならば、扱うデータもGPU側に置いておく方が高速
 - CPU-GPU間のデータ転送コストがボトルネックになることも
(NVIDIA GH200などではCPU-GPU間の帯域が太いため、状況を緩和)



- (NVIDIA GPUの場合の)GPUメモリの使い方は3通り
 - Unified Memory: CPUとGPUどちらかのメモリにデータを置き、CPU・GPU双方から直接アクセス
 - Managed Memory: CPUとGPU両方のメモリにデータを置き、システムが必要に応じて自動同期
 - Separate: ユーザーが明示的にメモリ管理(GPUコンピューティング最初期からの手法であり、上記2手法は利便性向上のため後に追加された使い方)

Unified Memory

- CPUとGPUどちらかのメモリにデータを置き,
CPU・GPU双方から直接アクセス
 - NVIDIA GH200/GB200などの機種でサポートされている
 - nvc/nvc++では, `-gpu=mem:unified:nomanagedalloc`を指定
 - `:nomanagedalloc` なしでは次ページのmanagedが使われることもあるので, 追記を推奨
- `malloc/new` などで動的に確保したメモリに対して,
 - CPU・GPUのはじめにアクセスした側のメモリにデータを配置(ファーストタッチ)
 - 以降のアクセスパターンについても監視され, GPU側に置いた方が高速化されると判断されると(自動的に)データがGPU側に置かれる(auto migration)
 - 下図では, a, bともに`malloc`で確保し, aはCPUから, bはGPUから1st touch



Managed Memory

- CPUとGPU両方のメモリにデータを置いておき、システムが必要に応じて自動的にデータを同期
 - 一旦メモリを読み、ページフォルトがあれば(自分の側に最新のデータがなければ)ページ単位でデータを転送するという仕組み(配列丸ごとの自動コピーではない)
 - データを少し読んで必要に応じて細切れにして転送するため、便利だが最高性能にはならない
 - MPI通信においては、CPUメモリを経由してしまう
 - 全てのNVIDIA GPUでサポートされている
 - nvc/nvc++では、`-gpu=mem:managed`を指定
- `malloc/new` などで動的に確保したメモリを、CPU・GPU両側に配置
 - 下図では、`a`は`malloc`で動的に確保している(宣言・確保方法はCPUコードと同じ)



Separate

- ユーザーによる明示的なメモリ管理
- malloc/new などでCPU上に動的確保したメモリに対して, 各処理を記述
 - GPU側にもマッピング
 - CPUからGPUへのデータ転送
 - GPUからCPUへのデータ転送
 - メモリ解放

```
double *a;
a = (double*)malloc(sizeof(double) * size_t(N));
#pragma acc enter data create(a[0:N])
#pragma acc update device(a[0:N])
#pragma acc update host(a[0:N])
#pragma acc exit data delete(a[0:N])
free(a);
```



Section 3.

コンパイル方法

OpenACCコードのコンパイル方法など

- NVIDIA HPC SDKを使用
(下記はC++向け. C言語の場合はnvcを使用)
- Unified Memory 使用時のコンパイル方法
 - `$ nvc++ -acc=gpu -gpu=cc90,mem:unified:nomanagedalloc ¥
-Minfo=accel,opt`
 - リンク時にも `-acc=gpu -gpu=mem:unified:nomanagedalloc` を指定
 - `-gpu=cc90`は, 用いるGPUの世代(90はHopper(H100, GH200など))を指定
- Managed Memory 使用時のコンパイル方法
 - `$ nvc++ -acc=gpu -gpu=cc90,mem:managed -Minfo=accel,opt`
 - リンク時にも `-acc=gpu -gpu=mem:managed` を指定
- Separate 使用時のコンパイル方法(OpenACCのデフォルト)
 - `$ nvc++ -acc=gpu -gpu=cc90 -Minfo=accel,opt`
 - リンク時にも `-acc=gpu` を指定

コンパイルメッセージ例(-Minfo=accel,opt)

• GPUによる高速化に成功した例

```
nvc -O3 -acc=gpu -Minfo=accel,opt -gpu=cc90 -c diffusion.c
diffusion3d:
    12, Generating present(fn[:,f[:]])
    16, Loop is parallelizable
    18, Loop is parallelizable
    20, Loop is parallelizable
    Generating NVIDIA GPU code
    16, #pragma acc loop gang, vector(128) collapse(3) /*
blockIdx.x threadIdx.x */
    18, /* blockIdx.x threadIdx.x auto-collapsed */
    20, /* blockIdx.x threadIdx.x auto-collapsed */
```

- 並列化可能なループであると判定
 - Loop is parallelizable が出力
- GPU上で並列化している
 - Generating NVIDIA GPU code
以降の行が, 具体的にどうGPUにマッピングしたかの報告

• GPUによる高速化に失敗した例

```
nvc -O3 -acc=gpu -Minfo=accel,opt -gpu=cc90 -c diffusion.c
diffusion3d:
    12, Generating present(f[:])
    15, Loop carried dependence due to exposed use of f[:]
prevents parallelization
    Accelerator serial kernel generated
    Generating NVIDIA GPU code
    15, #pragma acc loop seq
    16, #pragma acc loop seq
    17, #pragma acc loop seq
    16, Loop carried dependence due to exposed use of f[:]
prevents parallelization
    17, Complex loop carried dependence of f-> prevents
parallelization
    Loop carried dependence due to exposed use of f[:]
prevents parallelization
```

- 並列化できないループであると検知
 - Loop carried dependence...
- GPU内で並列化できていない
 - serial kernel generated
 - #pragma acc loop seq

GPU上で実行されていることの確認方法

- GPU上で実行されているか簡易に確認する際に有用な環境変数がある
- NVIDIA HPC SDKを使用
 - NVCOMPILER_ACC_NOTIFY=1
 - GPU上でカーネルが実行されるたびに情報を出力
 - NVCOMPILER_ACC_NOTIFY=3
 - CPU-GPU間のデータ転送に関する情報も出力
 - NVCOMPILER_ACC_TIME=1
 - CPU-GPU間のデータ転送およびGPU上での実行時間を出力

Section 4.

GPU移植例

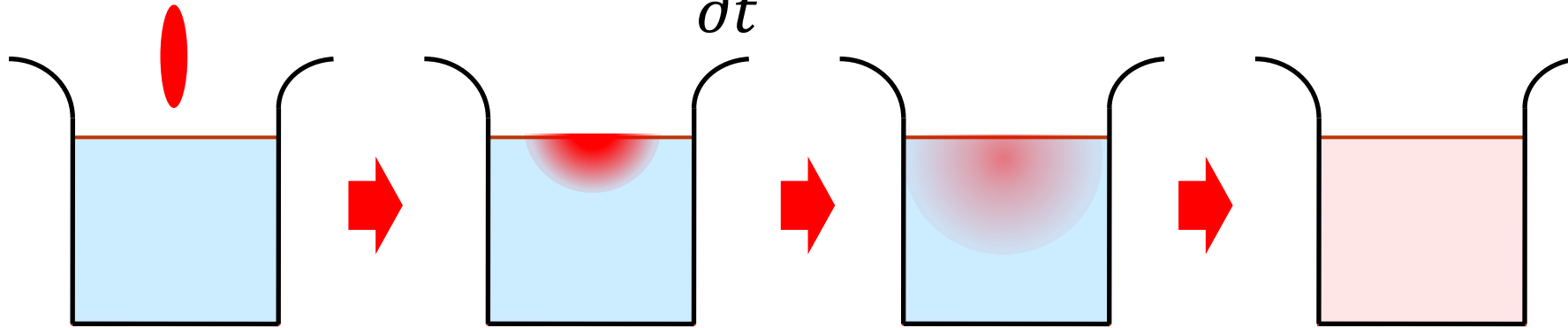
OpenACCによるGPU移植例

- 簡単な例を2つ紹介
 - 拡散方程式
 - 3次元一様格子の計算
 - メモリ律速な計算の代表例
 - N体問題
 - 粒子法の計算
 - 演算律速な計算の代表例

拡散方程式

- 物理量の拡散現象を計算(物理量 u , $a > 0$ は拡散係数):

$$\frac{\partial u}{\partial t} = a \nabla^2 u$$



- 一番シンプルな差分化を適用(簡単のため $h = \Delta x = \Delta y$ とする):
(空間2次元の場合には)

$$u_{i,j}^{n+1} = u_{i,j}^n + \frac{a \Delta t}{h^2} (u_{i,j-1}^n + u_{i,j+1}^n + u_{i-1,j}^n + u_{i+1,j}^n - 4u_{i,j}^n)$$

- 演算量に対して, メモリのロード・ストア量が多い(B/Fが大きい)
- 多重ループを結合(collapse)すると並列度を確保しやすい

拡散方程式のOpenACC実装例

・演算カーネル実装の抜粋

元のCPU実装

```
void diffusion3d(int nx, int ny, int nz, int mgn, float dx, float
dy, float dz, float dt, float kappa, const float *f, float *fn)
{
    #pragma omp parallel for
    for(int k = 0; k < nz; k++) {
        for (int j = 0; j < ny; j++) {
            for (int i = 0; i < nx; i++) {
                const int ix = nx * ny * (k + mgn) + nx * j + i;
                const int ip = i == nx - 1 ? ix : ix + 1;
                const int im = i == 0 ? ix : ix - 1;
                const int jp = j == ny - 1 ? ix : ix + nx;
                const int jm = j == 0 ? ix : ix - nx;
                const int kp = (k == nz - 1) ? ix : ix + nx*ny;
                const int km = (k == 0) ? ix : ix - nx*ny;

                fn[ix] = cc*f[ix] + ce*f[ip] + cw*f[im] +
                cn*f[jp] + cs*f[jm] + ct*f[kp] + cb*f[km];
            }
        }
    }
}
```

#pragma omp parallel for
を置き換え

OpenACC実装

```
void diffusion3d(int rank, int nx, int ny, int nz, int mgn, float dx,
float dy, float dz, float dt, float kappa, const float *f, float *fn)
{
    #pragma acc parallel
    #pragma acc loop independent collapse(3)
    for (int k = 0; k < nz; k++) {
        for (int j = 0; j < ny; j++) {
            for (int i = 0; i < nx; i++) {
                const int ix = nx * ny * (k + mgn) + nx * j + i;
                const int ip = i == nx - 1 ? ix : ix + 1;
                const int im = i == 0 ? ix : ix - 1;
                const int jp = j == ny - 1 ? ix : ix + nx;
                const int jm = j == 0 ? ix : ix - nx;
                const int kp = (k == nz - 1) ? ix : ix + nx * ny;
                const int km = (k == 0) ? ix : ix - nx * ny;

                fn[ix] = cc * f[ix] + ce * f[ip] + cw * f[im] + cn * f[jp] +
                cs * f[jm] + ct * f[kp] + cb * f[km];
            }
        }
    }
}
```

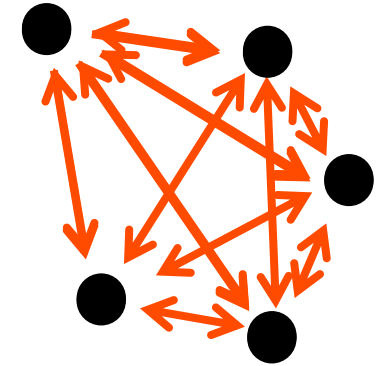
3重ループを結合し、
並列度を確保

N体計算(重力多体計算)

- 粒子どうしに働く自己重力による系の時間進化を、運動方程式に基づいて計算

- データ量: $\mathcal{O}(N)$
- 重力計算: $\mathcal{O}(N^2)$
- 時間積分: $\mathcal{O}(N)$

$$\mathbf{a}_i = \sum_{\substack{j=0 \\ j \neq i}}^{N-1} \frac{Gm_j (\mathbf{x}_j - \mathbf{x}_i)}{\left(|\mathbf{x}_j - \mathbf{x}_i|^2 + \epsilon^2\right)^{3/2}}$$



- 演算律速な問題の代表例
- (GPUの演算コアを埋められるほどに)十分な数の粒子数がある場合には、1つのスレッドが1つの粒子に働く重力を計算するように実装すると良い
 - 外側ループのみを並列化するような実装
- (N体計算の詳細については国立天文台CfCAの[N体学校](#)を参照)

N体計算のOpenACC実装例

• 演算カーネル実装の抜粋

```
static inline void calc_acc(const int32_t Ni, const float4
*const ipos, float4 *__restrict iacc, const int32_t Nj,
const float4 *const jpos, const float4 *const eps2) {
    #pragma omp parallel for
    for (int32_t i = 0; i < Ni; i++) {
        const auto pi = ipos[i];
        float4 ai = {0.0F, 0.0F, 0.0F, 0.0F};
        for (int32_t j = 0; j < Nj; j++) {
            const auto pj = jpos[j];
            // particle-particle interaction
            const auto dx = pj.x - pi.x;
            const auto dy = pj.y - pi.y;
            const auto dz = pj.z - pi.z;
            const auto r2 = eps2 + dx * dx + dy * dy + dz * dz;
            const auto r_inv = 1.0F / std::sqrt(r2);
            const auto r2_inv = r_inv * r_inv;
            const auto alp = pj.w * r_inv * r2_inv;
            // accumulation
            ai.x += alp * dx;
            ai.y += alp * dy;
            ai.z += alp * dz;
            ai.w += alp * r2;
        }
        iacc[i] = ai;
    }
}
```

元のCPU実装

```
static inline void calc_acc(const int32_t Ni, const float4 *const ipos,
float4 *__restrict iacc, const int32_t Nj, const float4 *const jpos,
const float4 *const eps2) {
    #pragma acc parallel
    #pragma acc loop independent
    for (int32_t i = 0; i < Ni; i++) {
        const auto pi = ipos[i];
        float4 ai = {0.0F, 0.0F, 0.0F, 0.0F};
        for (int32_t j = 0; j < Nj; j++) {
            const auto pj = jpos[j];
            // particle-particle interaction
            const auto dx = pj.x - pi.x;
            const auto dy = pj.y - pi.y;
            const auto dz = pj.z - pi.z;
            const auto r2 = eps2 + dx * dx + dy * dy + dz * dz;
            const auto r_inv = 1.0F / std::sqrt(r2);
            const auto r2_inv = r_inv * r_inv;
            const auto alp = pj.w * r_inv * r2_inv;
            // accumulation
            ai.x += alp * dx;
            ai.y += alp * dy;
            ai.z += alp * dz;
            ai.w += alp * r2;
        }
        iacc[i] = ai;
    }
}
```

OpenACC実装

Section 5.

関連情報

本資料で触れなかった内容

- 今後作成・公開する中級編において解説する予定
 - ここではキーワードのみ紹介しておく
 - OpenACC(2.7)の公式資料
 - 仕様書: <https://www.openacc.org/sites/default/files/inline-files/OpenACC.2.7.pdf>
 - Reference Card: <https://www.openacc.org/sites/default/files/inline-files/API%20Guide%202.7.pdf>
- GPUカーネルの非同期実行
 - `async` を指示文に追加
 - `wait` 文で同期を取る
- ブロックあたりのスレッド数を示唆する方法
 - `vector_length(n)`や`vector(n)`を指示文に追加
- Atomic演算の実行
 - `atomic`指示文を利用

資料提供者一覧

- 五十音順, 敬称略
- 下川辺 隆史(東京大学 情報基盤センター)
 - Section 4
- 星野 哲也(名古屋大学 情報基盤センター)
 - Sections 1, 2
- 三木 洋平(東京大学 情報基盤センター)
 - Sections 1, 2, 3, 4
- 山崎 一哉(東京大学 情報基盤センター)
 - Section 2

資料作成者一覧

- 五十音順, 敬称略
- 三木 洋平(東京大学 情報基盤センター)
 - 2026/3/25版